Author: Suhas Suresha & Dewang Sultania

SECTION 1

From documents to searchable knowledge

- **Grounding vs. guessing:** Published knowledge must be retrieved, not invented, when users ask about returns, compliance, or product facts.
- **Granularity:** Whole files are too coarse; chunks must be small enough for one idea yet large enough to stay meaningful when embedded.
- **Embeddings:** Dense vectors encode meaning, letting "laptop return" match "electronics restocking fee" despite different wording.
- **Fast similarity:** Vector stores use approximate nearest neighbor indexes so million-scale corpora still answer in milliseconds.
- **Attribution:** Hits need text, score, document id, and metadata so workflows can cite sources and filter by audience or department.

SECTION 2

Indexes and isolation

- **Named corpora:** An index is a boundary with its own embedding model, chunking policy, and document collection—like separate libraries, not one shared pile.
- **Why isolate:** Support queries must not surface legal-only chunks; isolation is stronger than forgetting a filter on a global table.
- **Frozen vectors:** Embedding model and dimensions are fixed at create time; mixing models in one index corrupts similarity geometry.
- **Per-index tuning:** FAQs, statutes, and API specs can each choose chunk size, overlap, and strategy without forcing neighbors to adopt the same grain.
- **Lifecycle:** Create, list, get, and delete index operations manage empty containers first, then ongoing ingest; ownership metadata underpins access control.

SECTION 3

Ingestion pipeline (detect, extract, metadata, chunk, embed, lifecycle)

- **Stage sequence:** Format detection routes bytes to parsers; extraction yields sections with headings and pages; metadata merges automatic and caller fields.
- **Shared parsers:** One platform pipeline fixes PDF column bleed and HTML chrome once instead of duplicating fragile extractors per team.
- **Chunking choice:** Fixed, recursive, or structure-aware splitters trade overlap cost against recall at section boundaries.
- **Model Service embeds:** Chunks embed through the same provider abstraction used elsewhere, preserving fallback, versioning, and cost attribution.
- **Jobs and re-ingest:** Large uploads return async job ids; stable `document_id` with replace-on-reingest avoids orphan vectors when policies update.

SECTION 4

Vector storage and search

- **VectorStore ABC:** Insert, delete-by-document, delete-index, and search hide whether Postgres, Pinecone, or another engine sits underneath.
- **pgvector pattern:** A chunks table stores text, `vector(N)` embeddings, JSON metadata, and index scoping; cosine distance becomes a similarity score in SQL.
- **Scoped queries:** Every search is constrained by `index_name` plus optional metadata filters and score thresholds so weak matches never flood the prompt.
- **Orchestration:** The service loads index config, embeds the query with the ingest model, then delegates ranking to the store—workflows pass plain text only.
- **Backend choice:** Teams already on Postgres may start with pgvector; dedicated vector databases enter when scale or multi-region ANN demands it.

SECTION 5

Hybrid search and reciprocal rank fusion

- **Complementary legs:** Vectors excel at paraphrase; keyword/BM25 legs excel at error codes, SKUs, and statutory citations.
- **Same result shape:** Both paths return unified SearchResult rows so downstream fusion and logging stay consistent.
- **RRF merge:** Reciprocal Rank Fusion combines rankings with `1/(k+r)` scores instead of adding incomparable raw similarities.
- **hybrid_search:** One entry point runs vector and lexical retrieval in parallel, fuses, then trims to `top_k` inside a single index.
- **Honest degradation:** Backends without lexical support should fail clearly rather than silently calling a hybrid API that only ran vectors.

SECTION 6

Service contract and the retrieval flow

- **RPC surface:** Index management, document ingest/status, Search, and HybridSearch expose the same capabilities through protobuf and the SDK.
- **SDK ergonomics:** Developers call `platform.data.ingest` and `platform.data.hybrid_search` without importing database dialects.
- **RAG assembly:** Retrieval returns ranked chunks; the workflow injects them into the Model Service prompt alongside session history from Chapter 4.
- **End-to-end trace:** Gateway auth, session context, filtered index search, generation, and retrieval telemetry (latency, tokens, hit ids) close the observability loop.
- **Platform gatekeeping:** Allowed embedding models and index ownership catch cost and compliance mistakes at configuration time, not after silent bad search.